

Transformers – Encoders

Prof. Anderson Rocha

Roteiro da Aula

1. Motivação e visão geral do Encoder
2. Embeddings de entrada (tokenização $\rightarrow$ vetores)
3. Embeddings posicionais (senoidais + opção aprendida)
4. Atenção (Q, K, V e softmax)
5. Multi-Head Attention (MHA)
6. Bloco de Encoder completo (Add & Norm, FFN)
7. Resumo Rápido

1) Por que Encoders em Transformers?

Ideia: construir **representações contextuais** de cada token sem recorrência nem convolução.

Encoder: lê uma sequência $\Rightarrow$ produz vetores contextualizados (um por posição).

Benefícios:

- ▶ Paralelismo total na sequência (treino mais rápido);
- ▶ Contexto global via *self-attention*;
- ▶ Escalável a domínios além de texto (visão, áudio, multimodal).

2) Embeddings de Entrada (dimensão média)

Configuração dos exemplos da aula:

- ▶ Tamanho do embedding: $d_{\text{model}} = 8$
- ▶ Exemplo de sequência (3 tokens): ["o", "gato", "dorme"]
- ▶ Representaremos cada token como vetor em $\mathbb{R}^8$

Exemplo (valores pedagógicos):

$$\mathbf{x}_o = [0.10, 0.20, 0.00, 0.30, -0.10, 0.05, 0.10, 0.15]$$

$$\mathbf{x}_{\text{gato}} = [0.70, -0.20, 0.10, 0.40, 0.05, 0.10, -0.30, 0.20]$$

$$\mathbf{x}_{\text{dorme}} = [-0.10, 0.25, 0.40, 0.10, 0.00, 0.05, 0.30, 0.20]$$

Observação prática: na vida real, esses vetores vêm de uma tabela de embeddings aprendida (por exemplo, inicializada com WordPiece/BPE).

3) Embeddings Posicionais (senoidais)

Transformers não têm noção de ordem. Adicionamos um **vetor posicional** $\mathbf{p}_{pos} \in \mathbb{R}_{\text{model}}^d$ por posição e somamos:

$$\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i.$$

Definição (Vaswani et al., 2017):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

Para $d_{\text{model}} = 8$ (**i=0,1,2,3**): denominadores 1, 10, 100, 1000.

3) Posicionais (cálculo numérico rápido)

Posição $pos = 0$ (útil para mostrar a ideia):

$$\mathbf{p}_0 = [\sin 0, \cos 0, \sin 0, \cos 0, \sin 0, \cos 0, \sin 0, \cos 0] = [0, 1, 0, 1, 0, 1, 0, 1].$$

Posição $pos = 1$ (aprox.):

$$\sin(1) \approx 0.8415, \cos(1) \approx 0.5403$$

$$\sin(0.1) \approx 0.0998, \cos(0.1) \approx 0.9950$$

$$\sin(0.01) \approx 0.0100, \cos(0.01) \approx 0.99995$$

$$\sin(0.001) \approx 0.0010, \cos(0.001) \approx 0.9999995$$

$$\mathbf{p}_1 \approx [0.8415, 0.5403, 0.0998, 0.9950, 0.0100, 0.99995, 0.0010, 0.9999995].$$

Nota: também é comum usar **posicionais aprendidos**; para hoje, vamos focar no senoidal.

4) Self-Attention: Q, K, V (ideia)

Para cada posição, projetamos o embedding em três espaços:

$$\mathbf{q}_i = \mathbf{z}_i W^Q, \quad \mathbf{k}_i = \mathbf{z}_i W^K, \quad \mathbf{v}_i = \mathbf{z}_i W^V.$$

Juntando todas as posições (sequência com n tokens):

$$Q \in \mathbb{R}^{n \times d_k}, \quad K \in \mathbb{R}^{n \times d_k}, \quad V \in \mathbb{R}^{n \times d_v}.$$

Atenção escalonada:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Intuição: $QK^\top$ mede compatibilidade consulta-chave; softmax gera pesos; multiplicamos por V para agregar conteúdo.

4) Self-Attention: Exemplo Numérico (1 cabeça)

Escolha didática: $d_{\text{model}} = 8$, $\# \text{cabeças} = 2 \Rightarrow d_k = d_v = 4$ por cabeça.

Vamos calcular a **atenção da cabeça 1** para o token 2 = "gato" sobre a sequência [o, gato, dorme].

Definimos (para facilitar contas):

$$\mathbf{q}_{\text{gato}}^{(1)} = [1, 0, 1, 0], \quad \mathbf{k}_o^{(1)} = [1, 0, 1, 0], \quad \mathbf{k}_{\text{gato}}^{(1)} = [0, 1, 0, 1], \quad \mathbf{k}_{\text{dorme}}^{(1)} = [1, 1, 0, 0].$$

$$\mathbf{v}_o^{(1)} = [1, 0, 0, 0], \quad \mathbf{v}_{\text{gato}}^{(1)} = [0, 1, 0, 0], \quad \mathbf{v}_{\text{dorme}}^{(1)} = [0, 0, 1, 0].$$

Compatibilidades (produtos escalares):

$$s_o = \mathbf{q}_{\text{gato}}^{(1)} \cdot \mathbf{k}_o^{(1)} = 2,$$

$$s_{\text{gato}} = \mathbf{q}_{\text{gato}}^{(1)} \cdot \mathbf{k}_{\text{gato}}^{(1)} = 0,$$

$$s_{\text{dorme}} = \mathbf{q}_{\text{gato}}^{(1)} \cdot \mathbf{k}_{\text{dorme}}^{(1)} = 1.$$

Como $\sqrt{d_k} = \sqrt{4} = 2$, os **scores** escalonados: [1, 0, 0.5].

4) Self-Attention: Exemplo Numérico (continuação)

Pesos via softmax: com $[1, 0, 0.5]$:

$$e^1 \approx 2.7183, \quad e^0 = 1, \quad e^{0.5} \approx 1.6487, \quad Z \approx 2.7183 + 1 + 1.6487 = 5.367.$$

$$\alpha \approx [0.506, 0.186, 0.307].$$

Saída da cabeça 1 (para "gato"):

$$\mathbf{y}_{gato}^{(1)} = 0.506 \mathbf{v}_o^{(1)} + 0.186 \mathbf{v}_{gato}^{(1)} + 0.307 \mathbf{v}_{dorme}^{(1)} = [0.506, 0.186, 0.307, 0].$$

Leitura: para esse *query*, a cabeça 1 focou mais em "o" (0.506), depois "dorme" (0.307), e menos em "gato" (0.186).

5) Multi-Head Attention (MHA)

Ideia: várias cabeças em paralelo, cada uma com projeções distintas W_i^Q, W_i^K, W_i^V :

$$\text{head}_i = \text{Attn}(QW_i^Q, KW_i^K, VW_i^V), \quad \text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O.$$

Exemplo qualitativo (2 cabeças):

- ▶ Cabeça 1: tende a capturar *co-ocorrências locais* (ex.: determinante–substantivo: "o" $\leftrightarrow$ "gato").
- ▶ Cabeça 2: foca em *dependências semânticas* (ex.: sujeito–verbo: "gato" $\leftrightarrow$ "dorme").

Observação prática: W^O re-mistura as cabeças para voltar à dimensão d_{model} .

5) MHA: Mini-exemplo numérico da cabeça 2

Para contraste, defina (cabeça 2):

$$\mathbf{q}_{gato}^{(2)} = [0, 1, 0, 1], \mathbf{k}_o^{(2)} = [1, 1, 0, 0], \mathbf{k}_{gato}^{(2)} = [1, 0, 1, 0], \mathbf{k}_{dorme}^{(2)} = [0, 1, 1, 0],$$

$$\mathbf{v}_o^{(2)} = [0, 0, 1, 0], \mathbf{v}_{gato}^{(2)} = [0, 0, 0, 1], \mathbf{v}_{dorme}^{(2)} = [1, 0, 0, 0].$$

Compatibilidades para "gato":

$$s = [1, 1, 2] \Rightarrow \text{scores} = \frac{[1, 1, 2]}{\sqrt{4}} = [0.5, 0.5, 1.0].$$

softmax $\Rightarrow$ pesos $\approx [0.211, 0.211, 0.577]$.

$$\mathbf{y}_{gato}^{(2)} \approx 0.211 [0, 0, 1, 0] + 0.211 [0, 0, 0, 1] + 0.577 [1, 0, 0, 0] = [0.577, 0, 0.211, 0.211].$$

5) MHA: Concatenação e projeção final

Concatene as saídas das duas cabeças ($\text{dim } 4 + 4 = 8$):

$$\mathbf{y}_{gato}^{\text{concat}} = [0.506, 0.186, 0.307, 0, 0.577, 0, 0.211, 0.211].$$

Projeção de volta para $d_{\text{model}} = 8$:

$$\mathbf{y}_{gato}^{\text{MHA}} = \mathbf{y}_{gato}^{\text{concat}} W^O \quad (\text{não calcularemos } W^O \text{ no quadro}).$$

Mensagem didática: cabeças diferentes capturam **padrões complementares**; a projeção final aprende a combiná-los.

6) Bloco de Encoder (camada completa)

Cada encoder layer contém:

1. **MHA** (self-attention)
2. **Add & Norm** (residual + normalização)
3. **FFN** posição-a-posição: $\mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{ff}}} \rightarrow \mathbb{R}^{d_{\text{model}}}$
4. **Add & Norm** (residual + normalização)

FFN típica:

$$\text{FFN}(\mathbf{u}) = (\text{ReLU}(\mathbf{u}W_1 + \mathbf{b}_1)) W_2 + \mathbf{b}_2.$$

Regra prática: $d_{\text{ff}} \approx 4d_{\text{model}}$ (ex.: $d_{\text{model}} = 8 \Rightarrow d_{\text{ff}} \approx 32$, aqui faremos menor no quadro, só para ilustrar).

6) Bloco do Encoder: fluxo com "o gato dorme"

1. **Embeddings:** $\mathbf{x}_i \in \mathbb{R}^8$ para cada token;
2. **Posicionais:** $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$;
3. **MHA:** $Z = [\mathbf{z}_1; \mathbf{z}_2; \mathbf{z}_3] \rightarrow \text{MHA}(Z)$;
4. **Add & Norm:** $\tilde{Z} = \text{LayerNorm}(Z + \text{MHA}(Z))$;
5. **FFN:** $\hat{Z} = \text{FFN}(\tilde{Z})$;
6. **Add & Norm:** $Z' = \text{LayerNorm}(\tilde{Z} + \hat{Z}) \Rightarrow$ **saída do encoder layer.**

Empilhamos L camadas idênticas (com pesos distintos) $\Rightarrow$ *representações profundas*.

7) Resumo Rápido

- ▶ **Posicionais senoidais:** $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$,
 $PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$
- ▶ **Q, K, V:** $Q = XW^Q$, $K = XW^K$, $V = XW^V$
- ▶ **Atenção:** $\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$
- ▶ **MHA:** $\text{Concat}(\text{heads})W^O$
- ▶ **Encoder layer:** $\text{MHA} \rightarrow \text{Add\&Norm} \rightarrow \text{FFN} \rightarrow \text{Add\&Norm}$
- ▶ **Sugestões:** h cabeças; $d_k = d_v = d_{\text{model}}/h$; $d_{\text{ff}} \approx 4d_{\text{model}}$

Exercício

Para a sequência ["o", "gato", "dorme"]:

1. Calcule $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ com $d_{\text{model}} = 8$ (1–2 casas decimais).
2. Some $\mathbf{x}_i + \mathbf{p}_i$ para obter $\mathbf{z}_i$.
3. Use a cabeça 1 (definida) para computar os pesos para o "gato" e a saída confirmando os números.
4. Como outra cabeça poderia focar em relações diferentes?

Solução I

Tarefa: Para a sequência ["o", "gato", "dorme"] e $d_{\text{model}} = 8$:

1. Calcular os posicionais $\mathbf{p}_0, \mathbf{p}_1, \mathbf{p}_2$ (1 casa decimal).
2. Calcular $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$ (1 casa decimal).
3. Usar a **cabeça 1** definida para obter pesos de atenção e saída do token "gato".
4. Como outra cabeça pode focar em relações diferentes? R. Uma outra cabeça de atenção pode focar em relações diferentes porque cada cabeça tem seus próprios parâmetros e aprende a projetar os vetores de entrada em espaços distintos.

Solução II

1) Embeddings Posicionais (1 casa decimal)

Fórmulas (Vaswani et al. 2017):

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right).$$

Como $d_{\text{model}} = 8 \Rightarrow 10000^{2i/d_{\text{model}}} = 10^i$, os divisores são 1, 10, 100, 1000.

Para $pos = 0$:

$$\mathbf{p}_0 = [0, 1, 0, 1, 0, 1, 0, 1].$$

Para $pos = 1$ (aprox. 1 casa):

$$\sin(1) \approx 0.8, \cos(1) \approx 0.5, \sin(0.1) \approx 0.1, \cos(0.1) \approx 1.0,$$

$$\sin(0.01) \approx 0.0, \cos(0.01) \approx 1.0, \sin(0.001) \approx 0.0, \cos(0.001) \approx 1.0.$$

$$\mathbf{p}_1 \approx [0.8, 0.5, 0.1, 1.0, 0.0, 1.0, 0.0, 1.0].$$

Solução III

Para $pos = 2$ (aprox. 1 casa):

$$\sin(2) \approx 0.9, \cos(2) \approx -0.4, \sin(0.2) \approx 0.2, \cos(0.2) \approx 1.0,$$

$$\sin(0.02) \approx 0.0, \cos(0.02) \approx 1.0, \sin(0.002) \approx 0.0, \cos(0.002) \approx 1.0.$$

$$\mathbf{p}_2 \approx [0.9, -0.4, 0.2, 1.0, 0.0, 1.0, 0.0, 1.0].$$

Solução IV

2) Vetores combinados $\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i$ (1 casa)

Embeddings (arredondados para 1 casa):

$$\mathbf{x}_o = [0.1, 0.2, 0.0, 0.3, -0.1, 0.1, 0.1, 0.2]$$

$$\mathbf{x}_{\text{gato}} = [0.7, -0.2, 0.1, 0.4, 0.1, 0.1, -0.3, 0.2]$$

$$\mathbf{x}_{\text{dorme}} = [-0.1, 0.3, 0.4, 0.1, 0.0, 0.1, 0.3, 0.2]$$

Posição 0:

$$\mathbf{z}_0 = \mathbf{x}_o + \mathbf{p}_0 = [0.1, 1.2, 0.0, 1.3, -0.1, 1.1, 0.1, 1.2].$$

Posição 1:

$$\mathbf{z}_1 = \mathbf{x}_{\text{gato}} + \mathbf{p}_1 = [1.5, 0.3, 0.2, 1.4, 0.1, 1.1, -0.3, 1.2].$$

Posição 2:

$$\mathbf{z}_2 = \mathbf{x}_{\text{dorme}} + \mathbf{p}_2 = [0.8, -0.1, 0.6, 1.1, 0.0, 1.1, 0.3, 1.2].$$

Solução V

3) Cabeça 1 — Pesos de atenção e saída para "gato" (1 casa)

Definições didáticas da cabeça 1 (dim por cabeça $d_k = d_v = 4$):

$$\mathbf{q}_{gato}^{(1)} = [1, 0, 1, 0], \quad \mathbf{k}_o^{(1)} = [1, 0, 1, 0], \quad \mathbf{k}_{gato}^{(1)} = [0, 1, 0, 1], \quad \mathbf{k}_{dorme}^{(1)} = [1, 1, 0, 0].$$

$$\mathbf{v}_o^{(1)} = [1, 0, 0, 0], \quad \mathbf{v}_{gato}^{(1)} = [0, 1, 0, 0], \quad \mathbf{v}_{dorme}^{(1)} = [0, 0, 1, 0].$$

Compatibilidades (produtos escalares):

$$s_o = 2, \quad s_{gato} = 0, \quad s_{dorme} = 1.$$

Com $\sqrt{d_k} = 2$, **scores** escalonados: [1.0, 0.0, 0.5].

Softmax (aprox. 1 casa):

$$e^{1.0} \approx 2.7, \quad e^{0.0} = 1.0, \quad e^{0.5} \approx 1.6, \quad Z \approx 2.7 + 1.0 + 1.6 = 5.3.$$

$$\alpha \approx [2.7/5.3, 1.0/5.3, 1.6/5.3] \approx \boxed{[0.5, 0.2, 0.3]}.$$

Solução VI

Saída da cabeça 1 para "gato":

$$\mathbf{y}_{gato}^{(1)} = 0.5 \mathbf{v}_o^{(1)} + 0.2 \mathbf{v}_{gato}^{(1)} + 0.3 \mathbf{v}_{dorme}^{(1)} = \boxed{[0.5, 0.2, 0.3, 0.0]}.$$

Solução VII

4) Cabeça 2

Uma **segunda cabeça** pode ressaltar padrões diferentes (ex.: sujeito-verbo). No exemplo da aula, ela resultou em pesos próximos a $[0.2, 0.2, 0.6]$ para "gato", favorecendo "dorme". A saída correspondente (1 casa) foi algo como:

$$\mathbf{y}_{gato}^{(2)} \approx [0.6, 0.0, 0.2, 0.2].$$

Concatenação das 2 cabeças (dim $4 + 4 = 8$):

$$\mathbf{y}_{gato}^{\text{concat}} = [0.5, 0.2, 0.3, 0.0, 0.6, 0.0, 0.2, 0.2].$$

A projeção final W^O (não calculamos aqui) retorna à dimensão $d_{\text{model}} = 8$.

Conclusão: Cabeças diferentes *complementam-se*: uma pode focar em relações locais ("o" $\leftrightarrow$ "gato"), outra em dependências semânticas ("gato" $\leftrightarrow$ "dorme"). A projeção W^O aprende a recombinar essas pistas.

Apêndice A – Notas conceituais

Por que dividir em Q, K, V?

Separamos *consulta* (o que busco), *chave* (como sou indexado) e *valor* (o que carrego). Isso dá liberdade para a rede aprender papéis distintos por cabeça.

Por que o fator $\sqrt{d_k}$?

Sem o escalonamento, produtos QK^T crescem em variância com a dimensão e saturam a softmax.

Posicionais senoidais vs. aprendidos:

Senoidais generalizam para comprimentos não vistos; aprendidos podem se ajustar melhor ao domínio. Atualmente, *rotary embeddings*, ALiBi e variações também são usados.

Apêndice B – Máscaras e normalização

Máscara no encoder:

Tipicamente usamos máscara apenas para *tokens de preenchimento* (PAD), não para causalidade (isso é do decoder).

LayerNorm:

Aplicada por posição (eixo de características). Ajuda na estabilidade de treino ao normalizar a ativação antes/depois dos submódulos.

Apêndice C – Mini-FFN com números (opcional no quadro)

Escolha leve para ilustração (não realista): $d_{\text{model}} = 8$, $d_{\text{ff}} = 8$,

$$W_1 = I, \quad b_1 = \mathbf{0}, \quad W_2 = I, \quad b_2 = \mathbf{0}.$$

Daí $\text{FFN}(\mathbf{u}) = \text{ReLU}(\mathbf{u})$ (apenas para mostrar o fluxo).

Na prática, W_1, W_2 são densas e $d_{\text{ff}} \approx 4d_{\text{model}}$.

Apêndice D – Próxima aula (Decoders)

- ▶ Atenção causal (máscara triangular)
- ▶ Atenção cruzada (queries do decoder; keys/values do encoder)
- ▶ Teacher forcing e shift-right nas entradas
- ▶ Critério de treino: cross-entropy token a token